

Homework

Spark, MongoDB e Neo4j con MovieLens 25M

Dataset: MovieLens 25M — <https://grouplens.org/datasets/movielens/25m/>

L'homework si propone di far acquisire esperienza pratica con tre paradigmi di storage e processing per Big Data, applicati a un dataset reale di larga scala nel dominio dello streaming video. Il dataset è rilasciato da GroupLens Research (University of Minnesota) con licenza non commerciale. È uno dei benchmark più consolidati per sistemi di raccomandazione e analisi di rating su larga scala.

File e struttura

File	Dim.	Contenuto
ratings.csv	640 MB	25M righe: <code>userId</code> , <code>movieId</code> , <code>rating</code> (0.5–5.0), <code>timestamp</code>
movies.csv	1.8 MB	62.000 film: <code>movieId</code> , <code>title</code> , <code>genres</code> (pipe-separated)
tags.csv	20 MB	1M righe: <code>userId</code> , <code>movieId</code> , <code>tag</code> (stringa libera), <code>timestamp</code>
links.csv	2.6 MB	Mapping <code>movieId</code> → <code>imdbId</code> , <code>tmdbId</code>
genome-scores.csv	312 MB	15M righe: <code>movieId</code> , <code>tagId</code> , <code>relevance</code> score (0–1)
genome-tags.csv	18 KB	1.129 tag descrittivi con <code>tagId</code> e label

1 Esercizi

1.1 Parte A — Apache Spark

1.1.1 Distribuzione dei rating per genere

Calcolare, per ogni genere (Action, Drama, Comedy, ...), il numero totale di rating, il rating medio, la mediana e la deviazione standard. Ordinare per rating medio decrescente.

1.1.2 Evoluzione temporale del rating medio

Calcolare il rating medio mensile (anno-mese) aggregato su tutti i film. Identificare i 3 mesi con il rating medio più alto e i 3 con il più basso.

1.1.3 Utenti prolifici reviewer e bias personale

Identificare gli utenti nel top-5% per numero di rating. Per ciascuno calcolare il rating medio personale e la differenza rispetto al rating medio globale (*bias personale*).

1.2 Parte B — MongoDB

Progettare uno schema a documento per rappresentare film, tag e rating.

Schema di partenza suggerito (da adattare e motivare):

```
// Collection: movies
{ "_id": <movieId>,
  "title": "Toy Story (1995)", "year": 1995,
  "genres": ["Adventure", "Animation", "Children"],
```

```
"stats": { "count": 81491, "avg": 3.92, "std": 0.85 },
"topTags": [{ "tag": "pixar", "relevance": 0.97 }, ...] }
```

1.2.1 Film per paese e decade

Calcolare il numero di film per paese e per decade (1980s, 1990s, 2000s, 2010s).

1.2.2 Utenti attivi per genere

Costruire una aggregation pipeline che, per ogni genere, restituisca: numero di utenti unici, numero totale di rating, i 5 titoli più recensiti.

1.2.3 Trend mensile per genere

Aggregation pipeline che raggruppi i rating per (**genere**, **anno**, **mese**) e calcoli volume e rating medio.

1.3 Parte C — Neo4j

Modellare i dati come grafo e usare **Cypher** per tutte le query.

Schema minimo richiesto:

- Nodi: (:User {userId}), (:Movie {movieId, title, year}), (:Genre {name}), (:Tag {name})
- Relazioni: (:User)-[:RATED {rating, timestamp}]->(:Movie)
- Relazioni: (:Movie)-[:HAS_GENRE]->(:Genre)
- Relazioni: (:User)-[:TAGGED {timestamp}]->(:Movie) via (:Tag)

1.3.1 Film più recensiti

Trovare i 10 film con il maggior numero di rating ricevuti.

1.3.2 Rating medio per genere

Calcolare il rating medio per ciascun genere, ordinato dal più alto al più basso.

1.3.3 Utenti con gusti simili

Dato un utente target, trovare i 5 utenti che hanno recensito il maggior numero di film in comune con rating ≥ 4.0 .